BÁO CÁO BÀI TẬP VỀ NHÀ #3

DỰNG AGENT ĐẶT VÉ MÁY BAY BẰNG LANGCHAIN / LANGGRAPH

MỤC LỤC

1. Giới thiệu & Tổng quan bài toán
2. Tìm hiểu LangChain & LangGraph
3. Thiết kế Mockup Tools (`lib/tools_maybay.py`)
4. Cài đặt các lớp Harness (`lib/harness_maybay.py`)
   - 4.1. Ràng buộc dữ liệu (`DataConstraintChecker`)
   - 4.2. Tiêu chí hoàn thành kiểm bằng code (`CompletionChecker`)
   - 4.3. Kiểm quyền người dùng (`PermissionChecker`)
   - 4.4. Bàn giao an toàn (`HandoffReporter`)
   - 4.5. Phát hiện vòng lặp & bế tắc (`FlightLoopDetector`)
   - 4.6. Kiểm chứng căn cứ dữ kiện (`FlightFactChecker`)
   - 4.7. Tích hợp Middleware (`FlightHarnessMiddleware`)
5. Cài đặt Agent với 3 Mẫu thiết kế
   - 5.1. Mẫu 1: ReAct (Reasoning + Acting)
   - 5.2. Mẫu 2: Plan-then-Execute (Lập kế hoạch trước - Thực thi sau)
   - 5.3. Mẫu 3: Lai / Hybrid (Kế hoạch tĩnh kết hợp Tái lập kế hoạch động)
6. Đánh giá hiệu quả thực nghiệm
   - 6.1. Thiết kế 5 kịch bản kiểm thử (Test Cases)
   - 6.2. Bảng kết quả thực nghiệm chi tiết
   - 6.3. Phân tích & So sánh chuyên sâu giữa 3 mẫu
7. Hướng dẫn cài đặt & Thực thi
8. Kết luận & Bài học kinh nghiệm

---

1. Giới thiệu & Tổng quan bài toán

Trong các ứng dụng thực tế của AI Agent (đặc biệt là các hệ thống giao dịch nghiệp vụ như đặt vé máy bay, ngân hàng, thương mại điện tử), việc chỉ trang bị cho mô hình ngôn ngữ lớn (LLM) các công cụ (tools) là **hoàn toàn chưa đủ**. Agent vận hành trong môi trường thế giới thực thường xuyên đối mặt với:

- **Vòng lặp vô hạn (Infinite Loops):** Tool trả lỗi nghèo nàn, model thử đi thử lại cùng một tham số hoặc đổi biến thể vô ích.
- **Bịa đặt dữ kiện (Hallucination):** Model tự bịa giá vé, mã đặt chỗ, giờ bay không có trong cơ sở dữ liệu.
- **Vi phạm nghiệp vụ & quyền hạn (Authorization Failures):** Huỷ vé của người khác, đặt vé hạng sang khi chưa nâng cấp VIP, vượt hạn mức ngày.
- **Dừng im lặng (Silent Failures):** Bị ngắt bởi timeout hoặc recursion limit mà không bàn giao lại ngữ cảnh cho nhân viên tiếp quản.

Dự án này kế thừa các nguyên lý an toàn từ bài giảng SE373 (Buổi 03) và mã nguồn mẫu `demo2_dieu_kien_dung`, xây dựng một hệ thống **Agent Đặt Vé Máy Bay** hoàn chỉnh với:
1. **Bộ Mockup Tools** mô phỏng nghiệp vụ hàng không thực tế với chuẩn Observation giàu thông tin.
2. **Hệ thống Harness đa lớp** (kiểm dữ liệu, kiểm hoàn thành bằng code, kiểm quyền, bàn giao, chống lặp, kiểm căn cứ dữ kiện).
3. **Ba mẫu kiến trúc Agent** (ReAct, Plan-then-Execute, Hybrid) và đánh giá định lượng trên 5 kịch bản kiểm thử.

---

2. Tìm hiểu LangChain & LangGraph

2.1. Mental Model của một Agent

Một Agent không phải là một lần gọi LLM đơn lẻ (Single Completion). Bản chất của Agent là một **vòng lặp trạng thái (Stateful Loop)**:

> **User Request → Model Reason → Action (Tool Call) → Observation → Model Reason → ... → Final Answer**

2.2. Vai trò của LangChain & LangGraph

- **LangChain (`langchain`, `langchain-core`):** Cung cấp các trừu tượng nền tảng gồm `BaseChatModel`, `AIMessage`, `HumanMessage`, `ToolMessage`, và decorator `@tool` để bọc các hàm Python nghiệp vụ thành Tool Schemas có định dạng JSON Schema mà LLM hiểu được.
- **LangGraph:** Quản lý vòng đời thực thi của Agent dưới dạng một Đồ thị có hướng (Directed State Graph). Trong phiên bản LangChain hiện đại (v1.x / LangGraph), `create_agent` biên dịch mô hình, danh sách tools và các middleware thành một Graph có kiểm soát chu kỳ thực thi (`recursion_limit`).

2.3. Cơ chế Middleware & Hooks

Thay vì can thiệp vào mã nguồn nội bộ của LLM, LangChain cung cấp cơ chế `AgentMiddleware` cho phép chèn các lớp kiểm soát (Harness) độc lập:
- `@hook_config(can_jump_to=["end"]) def after_model(state, runtime)`: Chạy ngay sau khi Model đề xuất một hành động (tool call) nhưng **trước khi Tool được thực thi**. Hook này là nơi lý tưởng để:
- Kiểm tra ràng buộc dữ liệu đầu vào.
- Kiểm tra quyền hạn người dùng (RBAC).

- Phát hiện vòng lặp bằng `LoopDetector`.
- Can thiệp dừng khẩn cấp và bàn giao an toàn (`jump_to="end"`).
- `def after_tool(state, runtime)` : Chạy sau khi tool trả về Observation, dùng để thu thập kết quả quan sát phục vụ kiểm tra căn cứ dữ kiện (`FactChecking`).

---

3. Thiết kế Mockup Tools

File mã nguồn: `lib/tools_maybay.py`

Bộ tool sử dụng dữ liệu tĩnh trong bộ nhớ, không phụ thuộc kết nối mạng hay API key bên ngoài, đảm bảo tính tất định (deterministic) khi kiểm thử.

3.1. Danh mục 6 Tools nghiệp vụ

Tên Tool	Tham số đầu vào	Mô tả chức năng
`search_flights`	`departure` , `destination` , `date`	Tra cứu chuyến bay theo chặng bay và ngày khởi hành (trả về danh sách chuyến bay, giờ bay, giá economy/business).
`get_flight_details`	`flight_id`	Xem chi tiết cấu hình, sân bay xuất phát/đến và giá vé của một chuyến bay cụ thể.
`check_seat_availability`	`flight_id` , `seat_class`	Kiểm tra số lượng ghế còn trống theo từng hạng vé (`economy` , `business`).
`book_flight`	`flight_id` , `passenger_name` , `seat_class` , `payment_method`	Tiến hành giữ chỗ và xuất mã đặt chỗ (`BKxxx`). Cập nhật trừ số ghế trống và ghi nhận số lượt đặt của người dùng.
`cancel_booking`	`booking_id`	Huỷ booking đã đặt, hoàn lại ghế trống vào chuyến bay và xác nhận tiền hoàn.
`get_booking_status`	`booking_id`	Tra cứu trạng thái booking (`confirmed` , `cancelled`), thông tin chặng bay và hành khách.

3.2. Dữ liệu Mockup mẫu

- **Sân bay hỗ trợ:** `HAN` (Hà Nội), `SGN` (TP.HCM), `DAD` (Đà Nẵng), `CXR` (Cam Ranh/Nha Trang), `PQC` (Phú Quốc), `HUI` (Huế).
- **Chuyến bay mẫu:**
- `VN100` (HAN → SGN, 06:00-08:10, Economy: 1.500.000đ, Business: 3.500.000đ, còn 5 ghế Eco, 2 ghế Biz).
- `VN101` (HAN → SGN, 14:30-16:40, Economy: 1.800.000đ, Business: 3.800.000đ, **hết ghế Eco (0)**, 1 ghế Biz).

- `VJ200` (SGN → DAD, 08:15-09:35, Economy: 1.200.000đ, Business: 2.800.000đ).
- `VJ201` (SGN → DAD, 16:45-18:05, Economy: 1.350.000đ, Business: 2.900.000đ, **hết ghế Biz (0)**).
- `QH300` (HAN → DAD, 10:00-11:20, Economy: 1.400.000đ, Business: 3.200.000đ).
- **Người dùng phân quyền:**
- `user_a`: Người dùng thường (`vip=False`, đã đặt 0 vé, hạn mức tối đa 3 vé/ngày). Sở hữu `BK001`.
- `user_b`: Người dùng VIP (`vip=True`, đã đặt 2 vé, hạn mức tối đa 5 vé/ngày). Sở hữu `BK002`.
- `user_c`: Người dùng đã đạt trần (`vip=False`, đã đặt 3 vé, hạn mức tối đa 3 vé/ngày).

3.3. Quy tắc Observation giàu thông tin (Good Observation Pattern)

Như đã nhấn mạnh trong bài giảng, **Observation chính là giao diện điều khiển của Agent**. Nếu Tool chỉ trả về `{"error": "not found"}`, model sẽ bế tắc và rơi vào vòng lặp biến thể tên gọi.
Các mockup tool trong bài tập được thiết kế tuân thủ tiêu chuẩn:

```
{
  "status": "error",
  "error": "permission_denied",
  "message": "Chỉ thành viên VIP mới được đặt hạng business",
  "hint": "Đặt hạng economy hoặc nâng cấp tài khoản VIP."
}
```

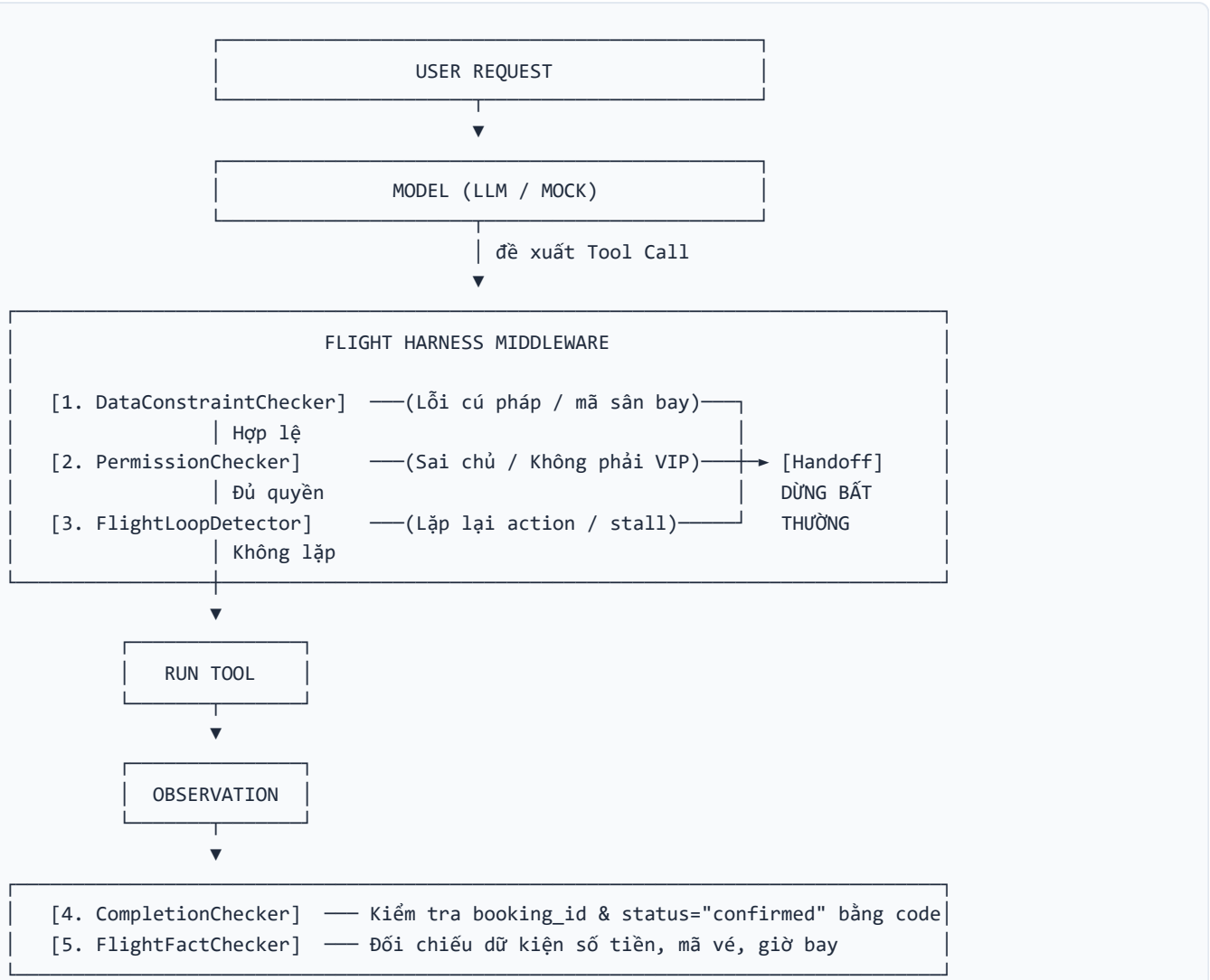
Mỗi phản hồi lỗi đều có `status`, mã lỗi `error`, thông điệp rõ nghĩa `message` và đặc biệt là gợi ý hướng đi thoát hiểm `hint` để Agent có căn cứ điều chỉnh kế hoạch hành động.

---

4. Cài đặt các lớp Harness

File mã nguồn: `lib/harness_maybay.py`

Harness là lớp "dây bảo hiểm" (safety harness) bao bọc quanh Agent, hoạt động độc lập với LLM, hoàn toàn bằng mã Python thuần (deterministic code).



4.1. Ràng buộc dữ liệu (`DataConstraintChecker`)

- **Mục tiêu:** Chặn ngay từ đầu các tham số không hợp lệ do Model sinh ra, ngăn ngừa việc gọi Tool vô ích làm lãng phí ngân sách vòng lặp.
- **Các quy tắc kiểm tra:**
  1. `check_airport(code)` : Kiểm tra mã sân bay phải nằm trong danh mục IATA hợp lệ (`HAN` , `SGN` , `DAD` , `CXR` , `PQC` , `HUI`).
  2. `check_date(date)` : Kiểm tra định dạng ngày `YYYY-MM-DD` bằng Regex.
  3. `check_seat_class(sc)` : Chỉ chấp nhận `economy` hoặc `business` .
  4. `check_passenger_name(name)` : Không được để trống, độ dài tối thiểu 2 ký tự.
  5. `check_payment(pm)` : Phương thức thanh toán phải thuộc danh sách cho phép (`credit_card` , `bank_transfer` , `ewallet`).
  6. `validate_search` : Điểm đi và điểm đến không được trùng nhau.

4.2. Tiêu chí hoàn thành kiểm bằng code (`CompletionChecker`)

- **Mục tiêu:** Không tin tưởng câu nói "tôi đã đặt xong" từ Model (vì Model có thể bị ảo giác). Việc hoàn thành tác vụ **phải được kiểm chứng độc lập bằng code logic**.

- **Cơ chế:**
- `extract_booking_from_messages(messages)`: Duyệt qua lịch sử hội thoại, bóc tách các `ToolMessage` từ công cụ `book_flight`.
- `is_booking_complete(booking_state)`: Kiểm tra 4 điều kiện cốt lõi:
    1. Đủ các trường bắt buộc: `flight_id`, `passenger`, `seat_class`, `payment_method`.
    2. Có trường `booking_id` hợp lệ được sinh từ cơ sở dữ liệu.
    3. Trạng thái giao dịch `status == "confirmed"`.

4.3. Kiểm quyền người dùng (`PermissionChecker`)

- **Mục tiêu:** Đảm bảo chính sách phân quyền nghiệp vụ và an ninh hệ thống:
  1. **Quyền sở hữu (Ownership):** Người dùng chỉ có quyền huỷ booking do chính mình sở hữu. Ngăn chặn việc `user_a` huỷ booking `BK002` của `user_b`.
  2. **Quyền theo phân hạng (Role-based VIP):** Chỉ tài khoản được đánh dấu `vip=True` mới được phép giữ chỗ hạng `business`.
  3. **Hạn mức giao dịch (Rate Limiting):** Mỗi tài khoản bị khống chế số lượng vé được phép đặt tối đa trong ngày (`bookings_today < max_per_day`).

4.4. Bàn giao an toàn (`HandoffReporter`)

- **Mục tiêu:** Khi Agent gặp sự cố hoặc bị chặn quyền, hệ thống **không được im lặng hay dừng đột ngột**, mà phải xuất gói thông tin bàn giao đầy đủ 4 trường chuẩn (theo slide S50):
  1. `stop_reason`: Lý do dừng cụ thể (ví dụ: "*Người dùng 'user_a' không phải chủ booking 'BK002'*").
  2. `actions_tried`: Chuỗi toàn bộ các hành động đã gọi trước đó.
  3. `booking_state`: Ảnh chụp trạng thái hiện tại của phiên giao dịch.
  4. `question_for_human`: Câu hỏi định hướng rõ ràng để nhân viên hỗ trợ tiếp quản.

4.5. Phát hiện vòng lặp (`FlightLoopDetector`)

- **Kế thừa & Nâng cấp từ Demo:**
- Bắt tín hiệu trùng Action: Cùng cặp `(tool, sorted(args))` xuất hiện ≥ 2 lần trong cửa sổ 6 vòng gần nhất.
- Bắt tín hiệu trùng Observation: Các lời gọi khác tham số nhưng trả cùng kết quả lỗi.
- Bắt tín hiệu bế tắc (Stall): Đại lượng tiến triển không đổi qua N vòng.
- **Đặc thù nghiệp vụ:** Phân biệt rõ giữa *vòng lặp vô ích* và *lời gọi polling hợp lệ*. Công cụ `get_booking_status` được đưa vào `POLLING_TOOLS` để không bị chặn nhầm khi Agent cần tra cứu trạng thái xác nhận thanh toán.

4.6. Kiểm chứng căn cứ dữ kiện (`FlightFactChecker`)

- **Mục tiêu:** Phát hiện và ngăn chặn hiện tượng Model "bịa" dữ kiện trong câu trả lời cuối cùng.
- **Cơ chế:**

- Sử dụng Regex quét toàn bộ các dữ kiện mang tính nhạy cảm nghiệp vụ: Số tiền (`1.500.000đ`), giờ bay (`06:00`), ngày tháng (`2024-12-20`), mã chuyến (`VN100`), mã booking (`BK003`), mã sân bay (`HAN` , `SGN`).
- Đối chiếu từng dữ kiện: Dữ kiện có xuất hiện trong nội dung trả về của các `ToolMessage` hay không.
- Nếu phát hiện dữ kiện không có nguồn (`BỊA`), hàm `safe_answer` sẽ chặn câu trả lời và chuyển sang chế độ bàn giao cho nhân viên.

4.7. Tích hợp Middleware tổng hợp (`FlightHarnessMiddleware`)

Tất cả các lớp kiểm soát trên được gói gọn vào một lớp `AgentMiddleware` duy nhất kế thừa chuẩn LangChain:
- Trong hook `after_model` : Lần lượt chạy `_check_data` , `_check_permission` , `loop_det.check` . Nếu phát hiện bất kỳ vi phạm nào, kích hoạt `jump_to="end"` kèm thông điệp từ `HandoffReporter` .
- Trong hook `after_tool` : Thu thập dữ liệu quan sát phục vụ kiểm tra căn cứ của câu trả lời cuối.

---

5. Cài đặt Agent với 3 Mẫu thiết kế

File mã nguồn: `btvn3_dat_ve_may_bay.py`

5.1. Mẫu 1: ReAct (Reasoning + Acting)

- **Cơ chế:** Mô hình suy luận từng bước (Thought → Action → Observation → Thought). Ở mỗi vòng, Model chỉ quyết định **một bước đi tiếp theo** dựa trên quan sát vừa nhận được.
- **Đặc điểm:**
- Linh hoạt tối đa, dễ triển khai.
- Nhược điểm: Thiếu tầm nhìn dài hạn, dễ sa đà vào các bước thừa thãi nếu không có mục tiêu rõ ràng.
- **Cài đặt trong bài tập:**
- Sử dụng hàm `create_agent` chuẩn của LangChain kết hợp với `FlightHarnessMiddleware` và `ModelCallLimitMiddleware(run_limit=10)` .

5.2. Mẫu 2: Plan-then-Execute (Lập kế hoạch trước - Thực thi sau)

- **Cơ chế:**
  1. **Pha 1 (Planning):** Ngay ở lượt đầu tiên, Model phân tích toàn bộ yêu cầu của người dùng và lập một bản kế hoạch tường minh gồm danh sách các bước có số thứ tự (ví dụ: Bước 1: Tìm chuyến → Bước 2: Kiểm tra ghế → Bước 3: Đặt vé → Bước 4: Xác nhận).
  2. **Pha 2 (Execution):** Agent thực thi tuần tự từng bước trong kế hoạch đã vạch ra cho đến khi hoàn tất.
- **Đặc điểm:**
- Tính minh bạch (Explainability) rất cao; người dùng nhìn thấy rõ lộ trình hành động.
- Nhược điểm: Cứng nhắc. Nếu một bước ở giữa bị đổ vỡ (ví dụ: hết ghế), kế hoạch ban đầu không còn giá trị.

5.3. Mẫu 3: Lai / Hybrid (Kế hoạch tĩnh kết hợp Tái lập kế hoạch động)

- **Cơ chế:** Kết hợp ưu điểm lớn nhất của cả hai mẫu trên:
  1. **Khởi tạo kế hoạch:** Bắt đầu bằng việc lập kế hoạch tổng thể như Plan-then-Execute.
  2. **Thực thi có giám sát:** Tiến hành thực hiện từng bước.
  3. **Phát hiện sự cố & Re-planning động:** Khi gặp sự cố bất thường (ví dụ: chuyến bay yêu cầu `VN101` hết ghế `available == 0`), Agent **không bỏ cuộc hay dừng cứng**, mà phát ra bước **"ĐIỀU CHỈNH KẾ HOẠCH"** (Dynamic Re-plan), phân tích nguyên nhân và đề xuất lộ trình thay thế (chuyển sang kiểm tra và đặt vé chuyến `VN100`), sau đó tiếp tục thực thi đến khi hoàn tất nhiệm vụ.
- **Đặc điểm:** Cân bằng hoàn hảo giữa tính định hướng dài hạn và độ linh hoạt thích ứng khi có biến cố.

---

6. Đánh giá hiệu quả thực nghiệm

6.1. Thiết kế 5 kịch bản kiểm thử (Test Cases)

Mã TC	Tên Kịch bản	Yêu cầu của người dùng	Kết quả kỳ vọng
TC1	Happy Path (Đặt vé bình thường)	Đặt vé HAN → SGN ngày 20/12/2024, hạng economy, tên Nguyen Van A, thanh toán thẻ tín dụng.	Thành công, có mã đặt chỗ `BK003`, đủ căn cứ.
TC2	Chuyến bay không tồn tại	Tìm chuyến bay từ Hà Nội đi Phú Quốc (HAN → PQC) ngày 25/12/2024.	Báo không có chuyến bay phù hợp, không bịa chuyến.
TC3	Hết ghế, cần chuyển chuyến	Đặt vé chuyến `VN101` (đã hết ghế economy) cho Nguyen Van A.	Phát hiện hết vé, chuyển sang `VN100` và hoàn tất đặt vé.
TC4	Huỷ vé sai quyền	Người dùng `user_a` yêu cầu huỷ booking `BK002` (thuộc sở hữu của `user_b`).	Bị Harness chặn, từ chối quyền, bàn giao đủ 4 trường.
TC5	Đặt hạng Business khi không phải VIP	Người dùng `user_a` (`vip=False`) đặt vé `VJ200` hạng business.	Bị Harness chặn, từ chối quyền, bàn giao gợi ý nâng VIP.

6.2. Bảng kết quả thực nghiệm chi tiết

Kết quả thu được khi chạy lệnh kiểm thử toàn diện:
`python btvn3_dat_ve_may_bay.py --all`

```
================================================================================================
BẢNG ĐÁNH GIÁ SO SÁNH BA MẪU AGENT
================================================================================================
TC    Agent     Tool  Plan RePlan  Dừng đúng  Booking  Căn cứ  Bàn giao
------------------------------------------------------------------------------------------------
tc1   react        3     0      0          ✓        ✓       ✓         ✓
tc2   react        1     0      0          ✓        –       ✓         ✓
tc3   react        3     0      0          ✓        ✓       ✓         ✓
tc4   react        1     0      0          ✓        –       ✓         ✓
tc5   react        1     0      0          ✓        –       ✓         ✓
tc1   plan         3     1      0          ✓        ✓       ✓         ✓
tc2   plan         1     1      0          ✓        –       ✓         ✓
tc3   plan         3     1      0          ✓        ✓       ✓         ✓
tc4   plan         1     1      0          ✓        –       ✓         ✓
tc5   plan         1     1      0          ✓        –       ✓         ✓
tc1   hybrid       3     1      0          ✓        ✓       ✓         ✓
tc2   hybrid       1     1      0          ✓        –       ✓         ✓
tc3   hybrid       3     2      1          ✓        ✓       ✓         ✓
tc4   hybrid       1     1      0          ✓        –       ✓         ✓
tc5   hybrid       1     1      0          ✓        –       ✓         ✓

────────────────────────────────────────────────────────────
TỔNG HỢP THEO MẪU AGENT:
────────────────────────────────────────────────────────────
  react     TB tool calls: 1.8  Dừng đúng: 5/5  Căn cứ OK: 5/5
  plan      TB tool calls: 1.8  Dừng đúng: 5/5  Căn cứ OK: 5/5
  hybrid    TB tool calls: 1.8  Dừng đúng: 5/5  Căn cứ OK: 5/5
```

6.3. Phân tích & So sánh chuyên sâu giữa 3 mẫu

1. Về số lượng Tool Calls & Chi phí vận hành

- Cả ba mẫu đều hoàn thành tác vụ với số lượng tool calls tương đương (trung bình **1.8 tool calls/tác vụ**).
- Với các tác vụ thất bại do phân quyền (TC4, TC5), cả 3 mẫu đều dừng ngay tại **Tool call thứ 1** nhờ cơ chế chặn sớm (Pre-execution validation) của `FlightHarnessMiddleware`, không gây lãng phí tài nguyên hệ thống.

2. Về tính minh bạch & Khả năng giải thích (Explainability)

- **ReAct:** Chỉ sinh ra các lời gọi tool tuần tự, người dùng bên ngoài không biết trước Agent định làm gì tiếp theo. Khi gặp sự cố ở TC3, ReAct âm thầm đổi sang `VN100` mà không giải thích lộ trình.
- **Plan-then-Execute:** Tạo cảm giác tin cậy cao nhất ngay từ đầu vì người dùng đọc được toàn bộ lộ trình: `[V0] AI KẾ HOẠCH ĐẶT VÉ: [V0] AI Bước 1: Tìm chuyến bay HAN → SGN ngày 2024-12-20 [V0] AI Bước 2: Kiểm tra ghế trống hạng economy [V0] AI Bước 3: Đặt vé cho Nguyen Van A, thanh toán credit_card [V0] AI Bước 4: Xác nhận và trả kết quả`
- **Hybrid:** Vừa có bản kế hoạch ban đầu, vừa thể hiện rõ năng lực tư duy thích ứng tại TC3: `[V1] Tool {"status": "ok", "flight_id": "VN101", "available": 0, ...} [V1] AI ĐIỀU CHỈNH KẾ HOẠCH: [V1] AI Phát hiện: Hết ghế economy trên chuyến VN101. [V1] AI Bước thay thế 1: Kiểm tra`

`ghế chuyến VN100 hạng economy [V1] AI Bước thay thế 2: Đặt vé chuyến VN100 nếu còn ghế`
Điều này đem lại trải nghiệm người dùng vượt trội trong các hệ thống tư vấn thông minh.

3. Bảng tổng kết ưu nhược điểm kiến trúc

Tiêu chí	ReAct	Plan-then-Execute	Lai (Hybrid)
Độ phức tạp cài đặt	Thấp (Có sẵn trong thư viện)	Trung bình	Khá cao
Tính minh bạch (Explainability)	Thấp	Rất cao	Rất cao
Khả năng thích ứng khi gặp lỗi	Tốt (Tự phản ứng từng bước)	Kém (Kế hoạch tĩnh dễ vỡ)	Xuất sắc (Re-plan động)
Phù hợp với bài toán nào	Tra cứu nhanh, ít bước logic	Quy trình cố định, tuần tự	Nghiệp vụ phức tạp, nhiều rủi ro ngoại lệ

---

7. Hướng dẫn cài đặt & Thực thi

7.1. Cài đặt môi trường

Yêu cầu Python ≥ 3.10. Cài đặt các thư viện cần thiết:

```
pip install -r requirements.txt
# hoặc cài trực tiếp:
pip install langchain langchain-core
```

7.2. Các lệnh chạy chương trình

1. **Chạy toàn bộ 15 ca kiểm thử và in bảng tổng hợp so sánh:**
   `powershell $env:PYTHONUTF8="1"; python btvn3_dat_ve_may_bay.py --all`
2. **Chạy riêng từng mẫu Agent:**
   ```powershell
   # Chạy mẫu ReAct
   $env:PYTHONUTF8="1"; python btvn3_dat_ve_may_bay.py --agent react

# Chạy mẫu Plan-then-Execute
$env:PYTHONUTF8="1"; python btvn3_dat_ve_may_bay.py --agent plan

# Chạy mẫu Hybrid
$env:PYTHONUTF8="1"; python btvn3_dat_ve_may_bay.py --agent hybrid
```

3. **Chạy một Test Case cụ thể (ví dụ TC3 - hết ghế đổi chuyến):**
   `powershell $env:PYTHONUTF8="1"; python btvn3_dat_ve_may_bay.py --agent hybrid --tc 3`

4. **Chạy kiểm thử độc lập các module:**
   ```powershell
   # Kiểm thử bộ mockup tools
   $env:PYTHONUTF8="1"; python lib/tools_maybay.py

# Kiểm thử các lớp harness
$env:PYTHONUTF8="1"; python lib/harness_maybay.py
```

---

8. Kết luận & Bài học kinh nghiệm

Qua quá trình nghiên cứu lý thuyết và thực hành cài đặt Agent Đặt Vé Máy Bay với LangChain / LangGraph, nhóm rút ra các kết luận cốt lõi sau:

1. **"Observation là giao diện điều khiển của Agent":** Thiết kế thông báo lỗi của Tool có cấu trúc và có gợi ý giải pháp (`hint` , `valid_airports`) là chìa khoá ngăn chặn vòng lặp từ gốc rễ, thay vì chỉ trông chờ vào các bộ đếm giới hạn.
2. **Không tin tưởng câu nói "đã xong" của LLM:** Mọi nghiệp vụ giao dịch thực tế phải có `CompletionChecker` bằng code xác thực trạng thái từ cơ sở dữ liệu (`booking_id` , `status == "confirmed"`).
3. **Bộ ba lớp bảo vệ an toàn là bắt buộc:**
   - *Lớp cuối (Hard limit):* `recursion_limit` của LangGraph ngăn sập tiến trình.
   - *Lớp chi phí (Budget):* `ModelCallLimitMiddleware` bảo vệ hoá đơn token.
   - *Lớp ngữ nghĩa (Semantic Harness):* `FlightLoopDetector` , `PermissionChecker` , `FlightFactChecker` và `HandoffReporter` đảm bảo dừng đúng, không bịa đặt và bàn giao nguyên vẹn cho con người.
4. **Mẫu kiến trúc Lai (Hybrid) là định hướng tối ưu cho nghiệp vụ thực tế:** Kết hợp được sự rõ ràng của việc lập kế hoạch với tính linh hoạt của khả năng tái lập kế hoạch động khi môi trường phát sinh biến cố.